

Podstawy Języka PHP

Rozpoczynanie i zakończenie poleceń PHP odbywa się za pomocą znaczników:

- Znacznik otwierający - `<?php`
- Znacznik zamykający - `?>`

Przykład 1

```
php

<?php
echo "<h2>Mój pierwszy skrypt PHP</h2>";
?>
```

Komentarze

Występują trzy rodzaje komentarzy:

- komentarz blokowy - zaczyna się od znaków `/*`, a kończy się `*/`
- komentarz jednowierszowy - zaczyna się od znaków `//` i kończy w bieżącej linii
- komentarz jednowierszowy uniksowy - zaczyna się od znaku `#` i ciągnie do końca linii

Zmienne

Przed nazwą zmiennej należy umieścić znak `$`. Warunki jakie muszą być spełnione przy nazwie:

- musi zaczynać się od litery lub znaku podkreślenia
- może składać się jedynie z liter, cyfr i znaku podkreślenia
- w nazwach rozróżniane są duże i małe litery
- w nazwach można stosować polskie litery

Przy odwoływaniu się do zmiennej zawsze należy użyć znaku `$`.

Przykład 2

```
php
```

```
<?php
$zmienna1=374;
$nazwa_1="to mój tekst";
$liczba7=26;
$ilosc=$zmienna1+$liczba7;
echo "<h3>Wynik wynosi $zmienna1<br> to jest nazwa $nazwa_1<br>$liczba7<br>Bież
?>
```

W języku PHP napisz skrypt, który będzie obliczał sumę, różnicę, iloraz, iloczyn i resztę z dzielenia dwóch dowolnych liczb.

```
php

<?php
$a=89;
$b=56;
$suma=$a+$b;
$roznica=$a-$b;
$iloczyn=$a*$b;
$iloraz=$a/$b;
$reszta=$a%$b;
echo "<h3>Suma wynosi: $suma<br> różnica wynosi: $roznica<br>Iloczyn wynosi: $
?>
```

Zmienne predefiniowane

- `$_GET[]` - jest to tablica zawierająca zmienne przesyłane do skryptu za pomocą metody GET
- `$_POST[]` - jest to tablica zawierająca zmienne przesyłane do skryptu za pomocą metody POST
- `$_COOKIE[]` - jest to tablica zawierająca zmienne przesyłane do skryptu za pomocą cookies()
- `$_FILES[]` - jest to tablica zawierająca zmienne przesyłane do skryptu podczas przesyłania plików na serwer
- `$_SERVER[]` - jest to tablica zawierająca zmienne przesyłane do skryptu przez serwer WWW. Są to dane takie jak wersja serwera, ścieżka do pliku, adres skryptu, wysłane nagłówki
- `$_ENV[]` - jest to tablica zawierająca wartości zmiennych środowiskowych serwera
- `$_REQUEST[]` - jest to tablica zawierająca zmienne przesyłane do skryptu przez użytkownika. Obejmuje dane z `$_GET`, `$_POST`, oraz `$_COOKIE`
- `$_SESSION[]` - jest to tablica zawierająca zmienne zarejestrowane w bieżącej sesji

- `$GLOBALS[]` - jest to tablica zawierająca odniesienie do każdej zmiennej utworzonej przez użytkownika, która ma zasięg globalny dla danego skryptu

Typy danych

Występujące w PHP typy danych można podzielić na trzy rodzaje:

1. Typy skalarne, czyli typy proste. Należą do nich:

- typ **boolean** - Jest to typ logiczny. Przyjmujący jedną z dwóch wartości: prawda (true) lub fałsz (false)
- typ **integer** - jest to typ liczb całkowitych. Może służyć do przedstawiania dodatnich lub ujemnych liczb całkowitych
 - Przykłady zapisu liczb całkowitych w różnych formatach:
 1. `537` - dodatnia liczba całkowita
 2. `-451` - ujemna liczba całkowita
 3. `032` - dodatnia liczba całkowita zapisana w formacie ósemkowym
 4. `-021` - ujemna liczba całkowita zapisana w formacie ósemkowym
 5. `0xFF` - dodatnia liczba całkowita zapisana w formacie szesnastkowym
 6. `-0x0C` - ujemna liczba całkowita zapisana w formacie szesnastkowym
- typ **float** lub **double** - jest to liczba zmiennie przecinkowa (rzeczywista)
przykład: 1,47; 0,17E2
- typ **string** - jest to typ łańcucha znaków, który służy do przedstawiania ciągu znaków. Pojedynczy znak ciągu jest zapamiętywany w jednym bajcie. Łańcuch znaków można utworzyć, korzystając z jednego z czterech sposobów:
 - używając znaków apostrofu
 - używając znaku cudzysłowu
 - używając składni *heredoc*

php

```
<?PHP
$napis='Dziś poprawia się pogoda';
$tekst=<<<TX
Tutaj rozpoznaję się $napis
TX;
echo $tekst;
?>
```

- używając składni *newdoc*

php

```
<?PHP
$napis="Dziś poprawia się pogoda";
$tekst=<<<'PC'
Tutaj rozpoznaję się $napis
PC;
echo $tekst;
?>
```

Funkcje języka PHP do obsługi bazy MySQLi

Funkcje biblioteki mysqli	Zwracana wartość
<code>mysqli_connect(serwer, użytkownik, hasło, nazwa_bazy)</code>	id połączenia lub FALSE, gdy niepowodzenie
<code>mysqli_select_db(id_polaczenia, nazwa_bazy)</code>	TRUE/FALSE w zależności od stanu operacji
<code>mysqli_error(id_polaczenia)</code>	Tekst komunikatu błędu
<code>mysqli_close(id_polaczenia)</code>	TRUE/FALSE w zależności od stanu operacji
<code>mysqli_query(id_polaczenia, zapytanie)</code>	Wynik zapytania
<code>mysqli_fetch_row(wynik_zapytania)</code>	Tablica numeryczna odpowiadająca wierszowi zapytania
<code>mysqli_fetch_array(wynik_zapytania)</code>	Tablica zawierająca kolejny wiersz z podanych w wyniku zapytania lub FALSE, jeżeli nie ma więcej wierszy w wyniku zapytania
<code>mysqli_num_rows(wynik_zapytania)</code>	Liczba wierszy w podanym zapytaniu
<code>mysqli_num_fields(wynik_zapytania)</code>	Liczba kolumn w podanym zapytaniu
<code>isset(\$zmienna)</code>	TRUE/FALSE w zależności od tego, czy \$zmienna istnieje

Główne różnice między \$_GET[] a \$_POST[]

W PHP to sposób przesyłania danych (URL vs. ciało żądania), ich widoczność (jawne vs. ukryte), ograniczenia długości (GET jest ograniczony, POST nie) oraz przeznaczenie (GET do pobierania, POST do wysyłania lub operacji modyfikujących dane).

\$_GET[]

Sposób przesyłania danych:

- Dane są dołączane jako parametry do adresu URL (np. `strona.php?klucz=wartosc`)
- Widoczność: Dane są jawne i widoczne w pasku adresu przeglądarki, co zwiększa ryzyko ich wykradzenia
- Ograniczenie długości: Metoda GET ma ograniczenie długości przesyłanych danych (zazwyczaj do około 255 znaków)
- Bezpieczeństwo: Nie nadaje się do przesyłania poufnych informacji, takich jak hasła, ponieważ są one widoczne w historii przeglądarki i logach serwera
- Zastosowania: Używana do wysyłania danych, które są pobierane, np. w wyszukiwarkach, przy zmianie strony w paginacji czy do odnośników

\$_POST[]

Sposób przesyłania danych:

- Dane są przesyłane w treści żądania HTTP, poza adresem URL, co zapewnia większą poufność
- Widoczność: Dane nie są widoczne w adresie URL ani w historii przeglądarki, co czyni je bardziej dyskretnymi
- Ograniczenie długości: Brak ograniczenia długości przesyłanych danych, co pozwala na wysyłanie dużych ilości informacji, w tym plików
- Bezpieczeństwo: Lepiej nadaje się do przesyłania poufnych danych i operacji modyfikujących dane, takich jak logowanie, wysyłanie danych do bazy czy przesyłanie plików
- Zastosowania: Używana do przesyłania danych w formularzach (np. podczas logowania), dłuższych tekstów, danych do bazy czy uploadu plików

Funkcja header()

W PHP służy do wysyłania surowych nagłówków HTTP do przeglądarki klienta, co pozwala na kontrolę nad komunikacją między serwerem a przeglądarką, np. do przekierowania użytkownika pod inny adres URL, ustawienia typu zawartości strony czy zarządzania sesjami. Kluczowe jest, aby wywoływać tę funkcję przed wysłaniem jakiegokolwiek innej treści (HTML, znaki białej przestrzeni), ponieważ nagłówki muszą być wysłane przed ciałem odpowiedzi HTTP.

Jak działa funkcja header()

- Wysyłanie nagłówków: Funkcja header() pozwala na wysłanie dowolnego, poprawnego nagłówka HTTP, który przeglądarka lub inny klient HTTP przetworzy
- Przekierowania: Jest najczęściej używana do przekierowania użytkownika na inny adres URL, na przykład po udanym zalogowaniu lub zmianie struktury strony
- Ustawianie typu zawartości: Pozwala na określenie typu zawartości przesyłanych danych (np. `Content-Type: application/json`), co jest kluczowe w komunikacji z innymi systemami lub aplikacjami
- Zarządzanie sesjami i pamięcią podręczną: Służy do zarządzania plikami cookie i ustawieniami pamięci podręcznej, co wpływa na działanie strony w przeglądarce

Przekierowanie na inny adres URL

```
php

header("Location: https://example.com");
exit();
```

Stałe w PHP

W Języku PHP występują stałe, czyli identyfikatory, których wartości nie ulegają zmianie. Do definiowania stałych służy funkcja `define()`, która ma dwa argumenty: nazwę stałej oraz przypisaną do niej wartość.

Definicja stałej ma postać:

```
php

define("NAZWA_STALEJ", wartość);
```

Przykład dla stałej

```
php

<?php
define("wiek", 21);
echo "<br>Nasz wiek to: ".wiek;
?>
```

Stałe predefiniowane

1. `PHP_VERSION` - stała reprezentująca aktualnie używaną wersję PHP
2. `PHP_OS` - stała zawierająca nazwę systemu operacyjnego, na którym uruchamiany jest PHP

3. `__FILE__` - stała zawierająca nazwę pliku ze skryptem, który jest aktualnie przetwarzany

Operatory i wyrażenia

W PHP operatory możemy podzielić na:

1. Arytmetyczne

- dodawanie: `+`
- odejmowanie: `-`
- mnożenie: `*`
- dzielenie: `/`
- reszta z dzielenia (modulo): `%`
- potęgowanie: `**`

2. Porównania

- argumenty są sobie równe: `==`
- argumenty są różne: `!=`
- argumenty są tego samego typu i są równe: `===`
- argumenty są różnego typu i są różne: `!==`
- argument pierwszy jest większy od drugiego: `>`
- argument pierwszy jest mniejszy od drugiego: `<`
- argument pierwszy jest większy lub równy od drugiego: `>=`
- argument pierwszy jest mniejszy lub równy od drugiego: `<=`
- argumenty są różne: `<>`

3. Bitowe

Umożliwiają wykonanie operacji na poszczególnych bitach liczb:

- iloczyn bitowy (AND): `&`
- suma bitowa (OR): `|`
- negacja bitowa (NOT): `~`
- bitowa różnica symetryczna: `^`
- przesunięcie bitowe w prawo: `>>`
- przesunięcie bitowe w lewo: `<<`

4. Logiczne

- iloczyn logiczny: `and`, `&&`
- suma logiczna: `or`, `||`
- negacja logiczna: `!`

- różnica symetryczna: `xor`

5. Przypisania

- `=`: `$x=23;`
- `+=`: `$x+=5;`
- `-=`: `$x-=5;`
- `*=`: `$x*=5;`
- `/=`: `$x/=5;`
- `%=`: `$x%=5;`

6. Łańcuchowe (znak konkatencji)

- łączenie łańcuchów znakowych: `.`: `$x="moje "." miasta";`
- dołączanie łańcucha znakowego: `.=`: `$x="moje "; $x.=" miasta ";`

php

```
<?php
```

```
$osoba["nazwisko"]="Kowalski";
```

```
$osoba["imie"]="Jan";
```

```
$osoba["wiek"]=27;
```

```
echo $osoba["nazwisko"]. " ".$osoba["imie"]." ma ".$osoba["wiek"]. " lat. ";  
?>
```

7. Inkrementacji i dekrementacji

- zwiększanie licznika o 1: `$x++`
- zmniejszanie licznika o 1: `$x--`

Funkcje daty i czasu

Język PHP dysponuje dużą grupą funkcji, których zadaniem jest wykonywanie operacji na dacie i czasie.

Funkcja `time()`

Zwraca informację na temat bieżącej daty i czasu. Nie ma żadnych argumentów. Informacje na temat daty i czasu są zwracane w postaci liczby. Odpowiada ona liczbie sekund, które upłynęły od godziny 00:00:00 1 stycznia 1970 roku do bieżącej daty. Jest to tzw. **znacznik czasu (timestamp)**.

php

```
<?php
echo time();
?>
```

Funkcja getdate()

Zwraca informacje dotyczące aktualnej daty i czasu jako tablica asocjacyjna.

Funkcja date()

`date()` zwraca datę sformatowaną zgodnie z szablonem przedstawionym w poniższej tabeli.

Znacznik	Znaczenie	Przykład
d	Dzień miesiąca w formacie z zerem na początku	01 - 31
D	Tekstowy opis angielskiej nazwy dnia, trzy litery	Mon, Tue
F	Pełny angielski opis dnia miesiąca, taki jak January czy March	January
g	Godzina, w formacie 12-godzinnym, bez zer wiodących	od 1 do 12
G	Godzina, w formacie 24-godzinnym, bez zer wiodących	od 1 do 24
H	Godzina, w formacie 24-godzinnym, z zerami wiodącymi	od 01 do 24
i	Minuty z zerami wiodącymi	od 01 do 59
l	Nazwa dnia tygodnia	Monday
m	Miesiąc w postaci liczby dwucyfrowej z zerem na początku	od 01 do 12
s	Liczba sekund z zerem na początku	od 01 do 59
Y	Rok w postaci czterech znaków	2023
w	Liczbowa forma dnia tygodnia	0 (dla Niedzieli) aż do 6 (dla Soboty)
W	Numer tygodnia w roku, zgodny z normą ISO-8601	Przykład: 42 (42. tydzień roku)

PHP - operacje na plikach

Funkcja fopen()

Funkcja otwierania pliku, ogólna składnia: `fopen('nazwa_pliku', 'tryb_otwarcia')`

Zestawienie trybów otwarcia

Tryb	Opis
------	------

<code>r</code>	Otwiera tylko do odczytu; umieszcza wskaźnik pliku na jego początku
<code>r+</code>	Otwiera do odczytu i zapisu; umieszcza wskaźnik pliku na jego początku
<code>w</code>	Otwiera tylko do zapisu; umieszcza wskaźnik pliku na jego początku i obcina plik do zerowej długości. Jeśli plik nie istnieje to próbuje go utworzyć
<code>w+</code>	Otwiera do odczytu i zapisu; umieszcza wskaźnik pliku na jego początku i obcina plik do zerowej długości. Jeśli plik nie istnieje to próbuje go utworzyć
<code>a</code>	Otwiera tylko do zapisu; umieszcza wskaźnik pliku na jego końcu. Jeśli plik nie istnieje to próbuje go utworzyć
<code>a+</code>	Otwiera do odczytu i zapisu; umieszcza wskaźnik pliku na jego końcu. Jeśli plik nie istnieje to próbuje go utworzyć
<code>x</code>	Tworzy i otwiera plik tylko do zapisu; umieszcza wskaźnik pliku na jego początku. Jeśli plik już istnieje, wywołanie <code>fopen()</code> nie powiedzie się, zwróci <code>FALSE</code> i wygeneruje błąd na poziomie <code>E_WARNING</code>
<code>x+</code>	Tworzy i otwiera plik do odczytu i zapisu; umieszcza wskaźnik pliku na jego początku. Jeśli plik już istnieje, wywołanie <code>fopen()</code> nie powiedzie się

Funkcja fputs()

Zapisuje do otwartego pliku (lub funkcja `fwrite()`).

Funkcja zatrzyma się na końcu pliku lub po osiągnięciu określonej długości, w zależności od tego co nastąpi wcześniej.

Ta funkcja zwraca liczbę bajtów zapisanych w przypadku powodzenia, `FALSE` w przypadku porażki.

`fputs()` funkcji jest alias `fwrite()` funkcji.

Składnia:

```
php
```

```
fputs(file, string, length)
```

Parametr	Opis
file	Wymagany. Określa otworzenie pliku do zapisu
string	Wymagany. Określa ciąg zapisu otwartego pliku
length	Opcjonalny. Określa maksymalną liczbę bajtów do zapisania

Funkcja fclose()

Do zamykania pliku służy funkcja `fclose()` zapisywana w postaci:

```
php  
  
fclose(deskryptor)
```

Deskryptor to wartość zwrócona przez funkcję `fopen()`.

Funkcja fgets()

Odczyt danych:

```
php  
  
fgets(deskryptor, ile_znaków)
```

Dołączanie plików

Duży plik można podzielić i dołączać poszczególne pliki. Służy do tego polecenie **include** lub **require**.

```
php  
  
include 'plik.php';  
require 'plik.php';
```

Szyfrowanie haseł w języku PHP

Proces haszowania (w przeciwieństwie do szyfrowania) wytwarza z wejścia wyjście, z którego nie można już wyprowadzić oryginalnego ciągu.

Dlatego dobrze nadaje się do ochrony wrażliwych ciągów znaków, haseł i sum kontrolnych.

Inną cechą funkcji haszujących jest to, że zawsze generują wyjścia o tej samej długości, a mała zmiana na wejściu zawsze całkowicie zmienia całe wyjście.

Funkcje haszowania

W PHP istnieje wiele funkcji hashowych, ważne z nich to:

1. **Bcrypt:** `password_hash()` - Najbardziej bezpieczne haszowanie haseł, wolne obliczeniowo, używa wewnętrznej soli i haszuje iteracyjnie
2. `md5()` - Bardzo szybka funkcja nadająca się do haszowania plików. Wyjście ma zawsze 32 znaki
3. `sha1()` - Szybka funkcja haszująca do haszowania plików, używana wewnętrznie przez Git do haszowania commitów. Dane wyjściowe mają zawsze 40 znaków
4. `hash()` - Funkcja `hash()` służy do hashowania ciągów znaków. Przyjmuje ona trzy parametry - dwa obowiązkowe i trzecie opcjonalny:
 - `$algo` (string) - oznacza algorytm, którym chcemy użyć do zahaszowania ciągu. Aby poznać listę obsługiwanych algorytmów, możemy skorzystać z bezparametrowej funkcji `hash_algos()`
 - `$data` (string) - jest to ciąg znaków, które chcemy zhaszować
 - `$raw_output` (bool, domyślnie false) - określa czy chcemy otrzymać wynik w postaci czystego stringu, czy w postaci binarnej

Pliki cookies w PHP

Czym są pliki cookies

To niewielkie pliki tekstowe wysyłane przez serwer lub skrypt do przeglądarki i umieszczane przez nią na dysku użytkownika. Pliki te są częścią specyfikacji protokołu HTTP i są wysyłane do przeglądarki w postaci nagłówka o nazwie `Set-Cookie`. Służą do identyfikacji użytkownika, zawierają między innymi nazwę serwera, datę wygaśnięcia pliku oraz informację na temat domeny i ścieżki dostępu do pliku.

Zasady działania plików cookies

- Po nawiązaniu połączenia serwer wysyła do przeglądarki nagłówek `Set-Cookie`, który zawiera plik cookie
- Przeglądarka zapisuje plik na dysku użytkownika
- Przy kolejnym połączeniu z serwerem przeglądarka wysyła na serwer przechowywany na dysku plik cookie

Tworzenie pliku cookie

W skryptach PHP pliki cookies tworzone są za pomocą funkcji `setcookie()`, która ma postać:

```
php
```

```
setcookie(nazwa, wartość, czas_trwania, ścieżka_dostępu, domena, bezpieczeństwo
```

Argumenty funkcji to:

1. **nazwa** - nazwa pliku
2. **wartość** - wartość, która będzie przechowywana w pliku cookie
3. **czas trwania** - czas po jakim plik zostanie usunięty przez przeglądarkę. Jest on podawany jako liczba sekund, które upłynęły od 1 stycznia 1970 r
4. **ścieżka dostępu** - ścieżka dostępu na serwerze dla pliku cookie
5. **domena** - domena w której plik cookie będzie dostępny
6. **bezpieczeństwo** - przyjmuje wartości true lub false. Ustawienie na true spowoduje, że plik będzie mógł być przesłany tylko przez bezpieczne połączenie HTTPS
7. **tylko_http** - nie będzie dostępne z poziomu skryptów

Usuwanie plików cookie

Aby samodzielnie usunąć plik cookie, można ustawić dla niego czas ważności, który już upłynął:

```
php
```

```
setcookie("pismo", "Na skróty", time()-100, "/", "localhost", 0);
```

Data ostatnich odwiedzin na stronie

```
php
```

```
$mies=2592000+time();
setcookie("wizyta", date("F jS - g:ia"), $mies);

if(isset($_COOKIE['wizyta'])){
    $ostatnia=$_COOKIE['wizyta'];
    echo "Witamy ponownie!<br> Ostatni raz odwiedziłeś nas: ".$ostatnia;
}else{
    echo "Witamy na naszej stronie";
}
```

Funkcje w PHP

Funkcja jest ciągiem instrukcji stanowiącym blok kodu, który może być wielokrotnie wykorzystywany w różnych programach lub w różnych miejscach programu. Funkcja jest wywoływana przez podanie jej nazwy i listy argumentów.

Definicja funkcji ma postać:

php

```
function nazwa($argument1, $argument2,...){  
    instrukcje;  
}
```

Nazwa funkcji nie może zaczynać się od znaku (\$).

Wyznaczanie zakresu losowanych liczb

W większości przypadków chcemy wyznaczyć przedział, z którego ma zostać wylosowana liczba. PHP udostępnia nam taką możliwość. Wystarczy podać dwa argumenty w wywołaniu funkcji `rand()`, minimalną i maksymalną wartość.

php

```
<?php  
echo rand()."<br>";  
/* od 0. Dla systemów z rodziny Windows wynosi ono 32768,  
   natomiast w systemach Unixowych 2147483647 */  
  
$i = rand(15,30);  
echo "Wylosowana liczba z zakresu (15 - 30) to $i";  
?>
```

Szukanie podciągu

Bardzo ważny element, przydatny podczas tworzenia różnego rodzaju ksiąg gości oraz forów internetowych. Niezbędny szczególnie wtedy, gdy wpisy nie są moderowane i od razu wyświetlają się na stronie. Najczęściej chcemy sprawdzić, czy dany wyraz występuje wewnątrz ciągu znaków. Przykładowo, gdy chcemy wprowadzić zakaz wpisywania wulgaryzmów.

Najczęściej stosowana funkcja, spełniająca powyższe zadanie, to `strpos(ciąg w którym szukamy, czego szukamy)`. Przyjmuje ona dwa argumenty, ciąg szukany oraz ciąg, w którym szukamy. W przypadku znalezienia wyrazu wewnątrz szukanego stringa, funkcja zwróci pozycję występowania wyrazu. Jeżeli natomiast podciąg nie zostanie znaleziony, funkcja zwróci wartość logiczną FALSE.

php

```
<?php
$wpisany_tekst = "Rozwijam swoją wiedzę w dziedzinie PHP.";

$czy = strpos($wpisany_tekst, "cholera");

if ($czy == FALSE) // nie znaleziono słowa cholera
    echo "Można wyświetlić: $wpisany_tekst.";
else // znaleziono szukany wyraz
    echo "Tekst zawiera wulgarne słownictwo.";
?>
```

Funkcja zamiany na wielkie litery

Bardzo często, podczas obsługi formularzy, zachodzi potrzeba zamiany na wielkie litery. Przykładowo, chcemy przechowywać w bazie danych loginy użytkowników. Wyczułone algorytmy sprawdzające rozróżniają małe i wielkie litery. Admin, ADMIN, admin i AdMin to cztery różne stringi. Żeby zabezpieczyć się przed przypadkami dodania dwóch tych samych nazw, różniących się tylko wielkością liter, musimy nauczyć się zamieniać je w jedną oraz drugą stronę.

Do zamiany wszystkich liter w ciągu na wielkie, używamy funkcji `strtoupper()`. Poniżej przykład zastosowania:

```
php

<?php
// przykładowy ciąg znaków zawierający małe oraz wielkie litery
$wyswietl = "Bardzo dobre placki";

// użycie funkcji strtoupper()
$duze_litery = strtoupper($wyswietl);

// wyświetli się napis:
// BARDZO DOBRE PLACKI
echo $duze_litery;
?>
```

Zamiana na małe litery

W analogiczny sposób możemy zamienić wszystkie znaki na małe litery. Służy do tego funkcja `strtolower()`. Poniżej przykład:

```
php
```

```

<?php
// przykładowy ciąg znaków zawierający małe oraz wielkie litery
$wyswietl = "BarDzo d0Bre PlaCki";

// użycie funkcji strtolower()
$male_litery = strtolower($wyswietl);

// wyświetli napis:
// bardzo dobre placki
echo $male_litery;

?>

```

Funkcje usuwania ciągów znaków

Do usunięcia białych znaków z początku lub końca ciągu można użyć jednej z trzech funkcji: `trim()`, `ltrim()`, `rtrim()`

1. `trim()` - usuwa podane znaki z początku i końca ciągu
2. `ltrim()` - usuwa podane znaki z początku ciągu
3. `rtrim()` - usuwa podane znaki z końca ciągu

Sprawdzanie liczby znaków w ciągu

Często chcemy zbadać, czy wpisywany tekst (np. post na forum) ma określoną długość. Możemy to osiągnąć używając funkcji `strlen()`. Poniżej przykład:

```

php

<?php
// przykładowy ciąg znaków
$wyswietl = "bardzo lubię placki";

// użycie funkcji strlen()
$ilosc = strlen($wyswietl);

if ($ilosc > 20)
    echo "Treść dłuższa niż 20 znaków.";
else
    echo "Treść zawiera 20 znaków lub mniej.<br>";
    echo $ilosc;

?>

```

Formatowanie tekstu

Do formatowania tekstu w postaci kolumny o określonej szerokości można wykorzystać funkcję `wordwrap()`. Dzieli ona ciąg podany jako argument na linie o maksymalnej długości 75 znaków. Do rozdzielenia linii domyślnie używany jest znak `\n`.

```
php

<?PHP
$tekst="Funkcja wordwrap() służy do formatowania tekstu w postaci kolumny o okr
echo wordwrap($tekst, 30, "<br>\n");
?>
```

Teoria do analizy pobierania pliku w JavaScript

Wyrażenie `nazwapliku.value.split('\\').pop()` w JavaScript służy do wyciągnięcia samej nazwy pliku z pełnej ścieżki dostępu (np. z pola "input type="file">).

Oto dokładne wyjaśnienie krok po kroku:

1. `nazwapliku.value` - Pobiera zawartość pola (string), która zazwyczaj wygląda tak: `C:\fakepath\dokument.pdf` lub `C:\Users\Nazwa\Documents\zdjecie.png`
2. `.split('\\')` - Dzieli ten ciąg znaków na tablicę w miejscach, gdzie występuje odwrotny ukośnik (`\`)
 - Przykład: `"C:\Folder\plik.txt"` zamienia się w tablicę `["C:", "Folder", "plik.txt"]`
3. `.pop()` - Usuwa i zwraca ostatni element z tej tablicy
 - Przykład: Z tablicy `["C:", "Folder", "plik.txt"]` metoda `.pop()` wyciąga `"plik.txt"`

Podsumowując: Kod ten zwraca nazwę pliku wraz z rozszerzeniem, ignorując ścieżkę folderów, która znajdowała się przed nim.

Uwaga: W nowoczesnych przeglądarkach, ze względów bezpieczeństwa, pole pliku często zwraca `C:\fakepath\nazwa_pliku.ext`, dlatego ta metoda jest popularnym sposobem na pobranie czystej nazwy.

Metoda GET i odnośniki

Najpopularniejszy dla metody GET sposób przekazywania parametrów dla żądania to przesyłanie ich w adresie url.

Ogólna składnia: `index.php?lang=pl¶m1=2`

Przesyłane parametry znajdują się w tablicy globalnej `$_GET`